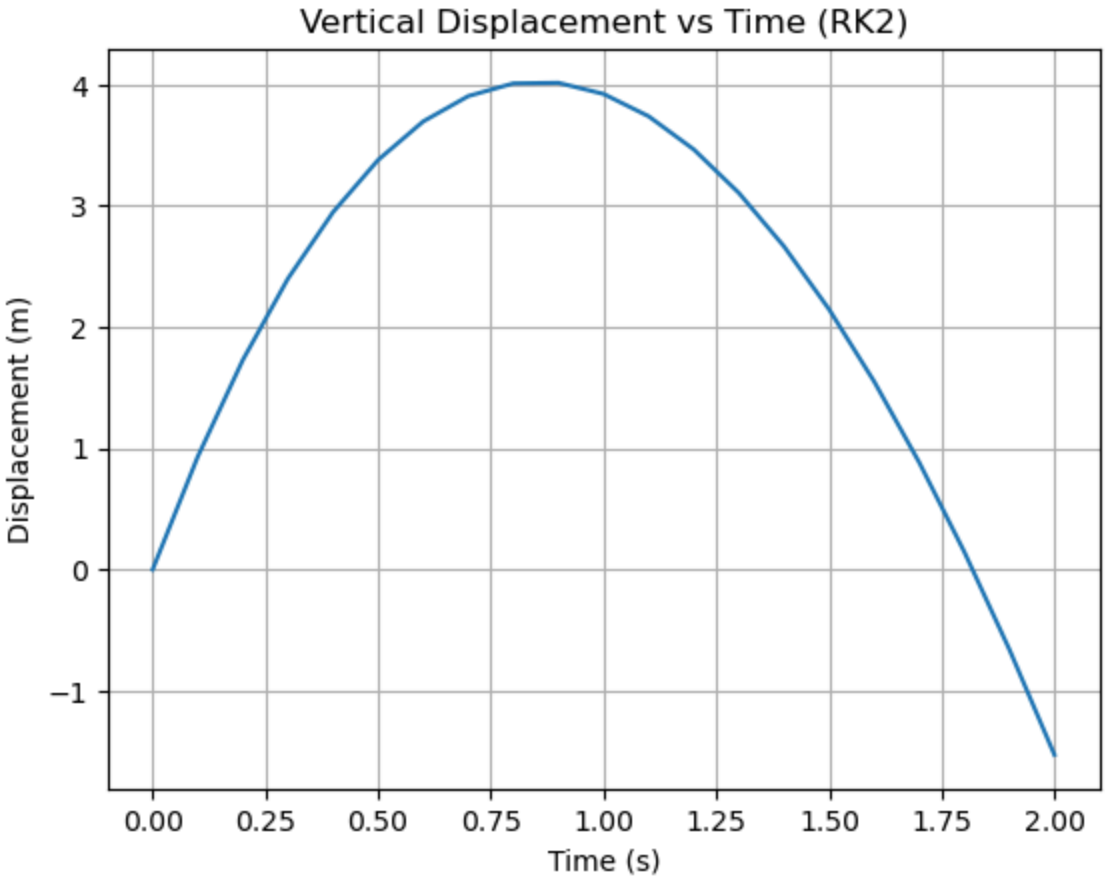
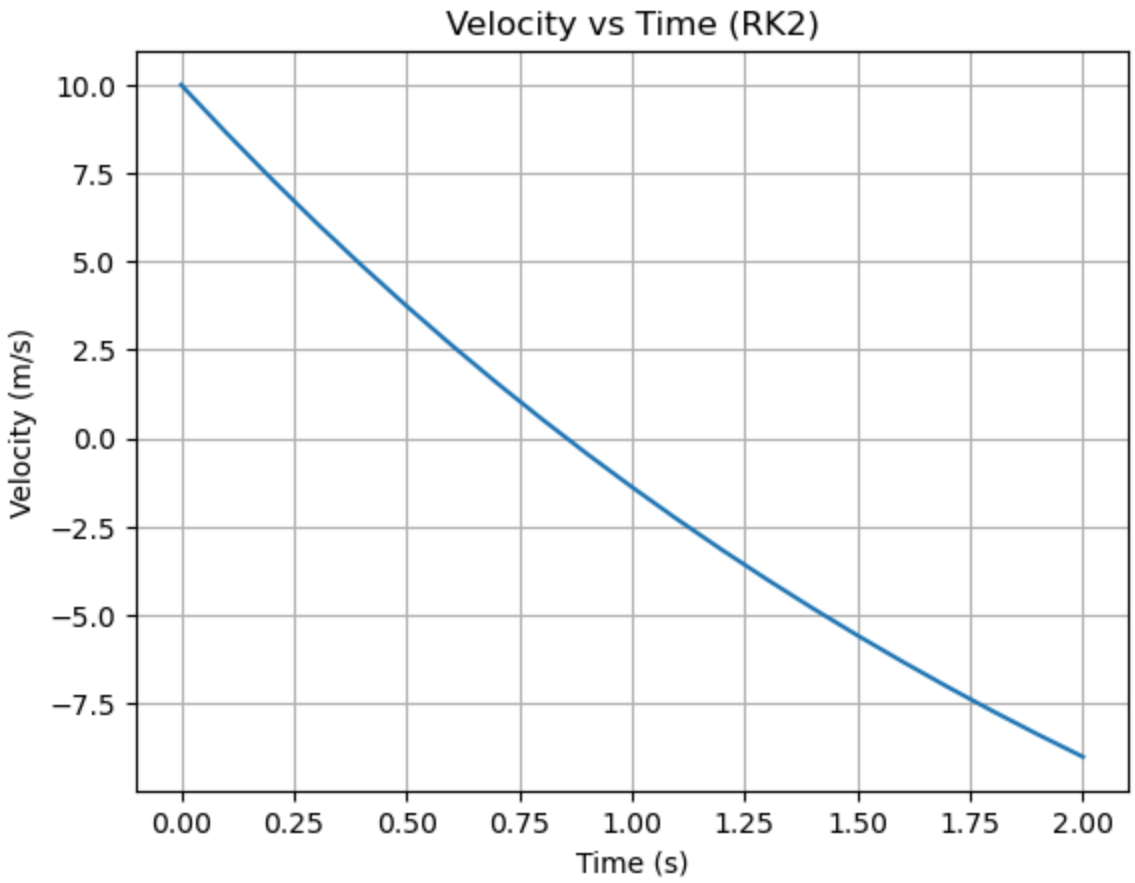
(a)

In [13]:

```
import numpy as np
import matplotlib.pyplot as plt

m = 0.5
k = 0.2
g = 9.81

t0 = 0
total_time = 2
h = 0.1
t_values = np.arange(t0, total_time + h, h)

y0 = 0
v0 = 10

def system_dynamics(t, y, v):
    dy_dt = v
    dv_dt = (-k * v - m * g) / m
    return dy_dt, dv_dt

def runge_kutta_second_order(f, y0, v0, t_values, h):
    y_values = np.zeros_like(t_values)
    v_values = np.zeros_like(t_values)

    y_values[0] = y0
    v_values[0] = v0

    for i in range(len(t_values) - 1):
        t = t_values[i]
        y = y_values[i]
        v = v_values[i]

        k1_y, k1_v = f(t, y, v)
        k2_y, k2_v = f(t + h, y + h * k1_y, v + h * k1_v)

        y_values[i + 1] = y + h / 2 * (k1_y + k2_y)
        v_values[i + 1] = v + h / 2 * (k1_v + k2_v)

    return y_values, v_values

y_values, v_values = runge_kutta_second_order(system_dynamics, y0, v0, t_values, h)

plt.plot(t_values, y_values, label="RK2 Displacement")
plt.xlabel('Time (s)')
plt.ylabel('Displacement (m)')
plt.title('Vertical Displacement vs Time (RK2)')
plt.grid(True)
plt.show()

plt.plot(t_values, v_values, label="RK2 Velocity")
```

```
plt.xlabel('Time (s)')
plt.ylabel('Velocity (m/s)')
plt.title('Velocity vs Time (RK2)')
plt.grid(True)
plt.show()

print(f"RK method second order result at t = {final_time_RK:.2f} : {final_voltage_R
```





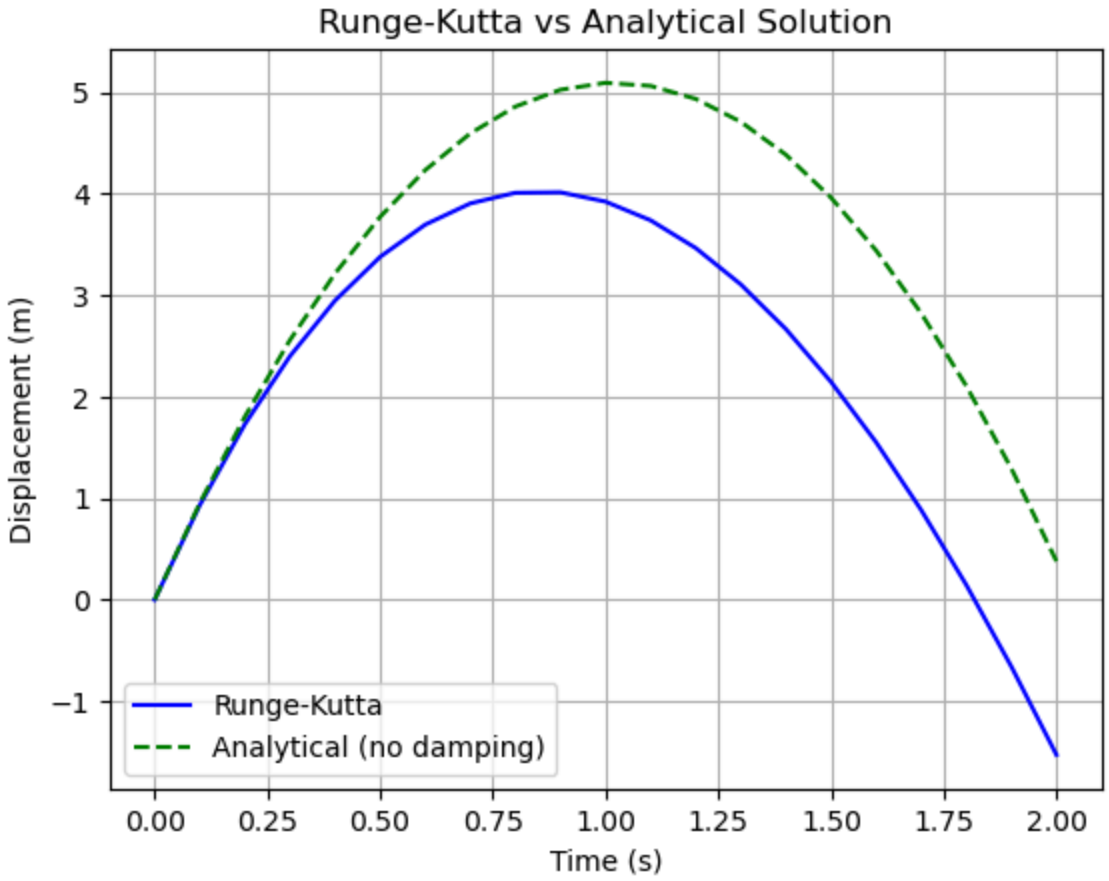

```
RK method second order result at t = 2.00 : -9.0119 m/s
```

(b)

In [14]:

```python
import numpy as np
import matplotlib.pyplot as plt

m = 0.5
k = 0.2
g = 9.81

t0 = 0
total_time = 2
h = 0.1
t_values = np.arange(t0, total_time + h, h)

y0 = 0
v0 = 10

def system_dynamics(t, y, v):
    dy_dt = v
    dv_dt = (-k * v - m * g) / m
    return dy_dt, dv_dt

def runge_kutta_second_order(f, y0, v0, t_values, h):
    y_values = np.zeros_like(t_values)
    v_values = np.zeros_like(t_values)
```

```
    y_values[0] = y0
    v_values[0] = v0

    for i in range(len(t_values) - 1):
        t = t_values[i]
        y = y_values[i]
        v = v_values[i]

        k1_y, k1_v = f(t, y, v)
        k2_y, k2_v = f(t + h, y + h * k1_y, v + h * k1_v)

        y_values[i + 1] = y + h / 2 * (k1_y + k2_y)
        v_values[i + 1] = v + h / 2 * (k1_v + k2_v)

    return y_values, v_values

y_values, v_values = runge_kutta_second_order(system_dynamics, y0, v0, t_values, h)

y_analytical = -0.5 * g * t_values**2 + v0 * t_values

plt.plot(t_values, y_values, 'b-', label='Runge-Kutta')
plt.plot(t_values, y_analytical, 'g--', label='Analytical (no damping)')
plt.xlabel('Time (s)')
plt.ylabel('Displacement (m)')
plt.title('Runge-Kutta vs Analytical Solution')
plt.legend()
plt.grid(True)
plt.show()
```


Runge-Kutta vs Analytical Solution
Displacement (m)
Time (s)
Runge-Kutta
Analytical (no damping)
5
4
3
2
1
0
−1
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00

In [ ]: